



VETRA

PUBLICATION SERIES • DOCUMENT 02

DOC ID: PUB-VETRA-DOC-02
MODULE: BACKEND & API
DATE: AUGUST 2026

• API & BACKEND ARCHITECTURE MONOGRAPH

VETRA

API & Backend Architecture

Engineering the Core Veterinary Operations Platform: Clean Architecture, Criteria Specifications, Security Filter Chains & Distributed Telemetry

ARCHITECTED & ENGINEERED BY

Om Rajput

B.Tech in Computer Science & Engineering (Data Science)
SVKM's Institute of Technology, Dhule

Backend Runtime & Framework

Java 21 LTS (Eclipse Temurin) • Spring Boot 3.5.16 • Spring Security 6 • Spring PostgreSQL 15.13 (PostGIS Spatial) • Redis 7.1 (Distributed Cache) • Data JPA • JWT 0.12.6

Persistence & Cache

Flyway 10 (V1-V14)

01 — CORE SYSTEM ROLE

Backend Engineering Overview

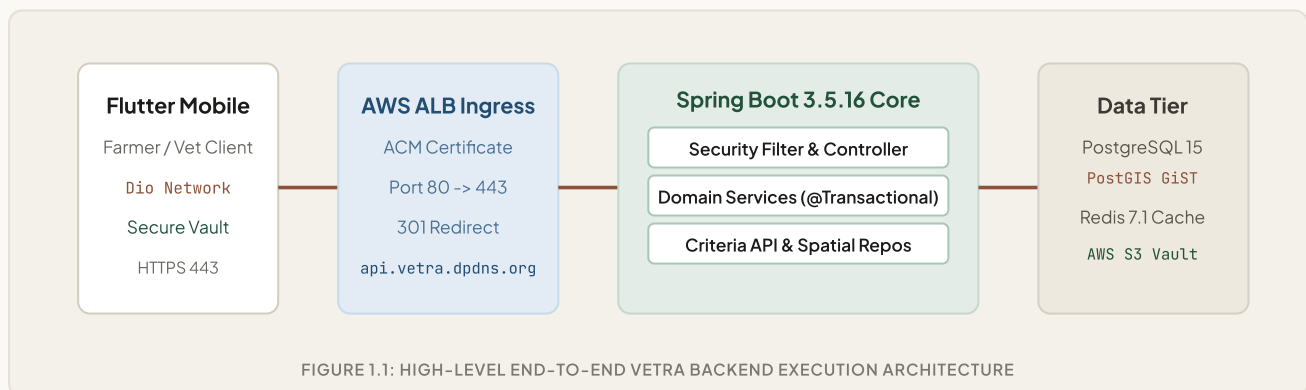
The central coordination engine governing clinical records, authentication invariants, spatial surveillance, and transactional boundaries.

Core Backend Responsibilities

1. **Deterministic Business Invariants:** Enforcing state machine transitions for appointments and immutable EVMR medical history.
2. **Zero-Trust Identity:** Stateless HMAC-SHA512 JWT verification, cryptographic refresh rotation, and tenant scoping.
3. **Geospatial Surveillance:** Executing PostGIS spatial queries (ST_DWithin) for real-time outbreak clustering.

Execution Context

The backend runs as containerized tasks on AWS ECS Fargate within private VPC subnets. It exposes versioned RESTful APIs (/api/v1) behind an AWS Application Load Balancer terminating TLS 1.3 on port 443 with automatic HTTP port 80 redirect.



Contract Immutability Guarantee

Every API response is encapsulated in a strongly typed `ApiResponse<T>` envelope. If a business violation occurs, a typed `BaseDomainException` triggers the `GlobalExceptionHandler`, ensuring consistent RFC 7807 error structures with zero unhandled stack traces leaked to clients.

02 — DEPENDENCIES

Backend Technology Stack & BOM Matrix

Verified production libraries, enterprise frameworks, and build coordinates configured in `pom.xml`.

COMPONENT / LAYER	ARTIFACT & VERSION	ARCHITECTURAL ROLE & PURPOSE
Runtime & Parent	spring-boot-starter-parent:3.5.16 Java 21 LTS (Eclipse Temurin)	Core enterprise foundation; provides modern virtual threads, pattern matching, and managed dependency convergence.
Web & Reactive	spring-boot-starter-web spring-boot-starter-webflux	Provides embedded Apache Tomcat servlet container and non-blocking WebClient for asynchronous external integrations.
Security & JWT	spring-boot-starter-security io.jsonwebtoken:jjwt-api:0.12.6	Stateless security filter chain with HMAC-SHA512 token parsing, BCrypt password hashing (strength 12), and method security.
Persistence & Spatial	spring-boot-starter-data-jpa hibernate-spatial:6.x postgresql:runtime	Hibernate 6 ORM with PostGIS spatial geometry mapping (Point coordinates, spatial queries, and GiST indexes).
Schema Migrations	flyway-core flyway-database-postgresql	Deterministic database version control executing 14 ordered SQL migration scripts (V1 through V14) at application startup.
Cache Infrastructure	spring-boot-starter-data-redis spring-boot-starter-cache	Multi-level distributed caching with Redis standalone/cluster topology, TTL expiration policies, and serialization.
Fault Tolerance	resilience4j-spring-boot3:2.4.0 resilience4j-micrometer:2.4.0	Circuit breaker state machines, rate limiters, and exponential backoff retry mechanisms for external system isolation.
Observability	micrometer-tracing-bridge-otel opentelemetry-exporter-otlp datasource-micrometer:1.0.6	Distributed tracing spans, JDBC connection pool telemetry via HikariCP, and OTLP span exports to Grafana Tempo.
Cloud Storage SDK	software.amazon.awssdk:s3:2.51.2 software.amazon.awssdk:sts:2.51.2	AWS SDK v2 client for presigned URL generation and secure binary asset storage in Amazon S3 buckets.

Dependency Convergence Strategy

All transient Spring dependencies are managed strictly via the Spring Boot 3.5.16 parent BOM and AWS SDK BOM (2.51.2), preventing jar hell and binary classpath conflicts.

Static Quality Enforcement

The build pipeline enforces Google Java Style standards via `maven-checkstyle-plugin:3.6.0` with 0 violations permitted, and automated Gitleaks SAST scanning on every commit.

03 — ARCHITECTURE

Modular Monolith & Package Boundaries

Enforcing inward-pointing Clean Architecture dependencies and bounded context isolation in Java 21.

1. Presentation Layer

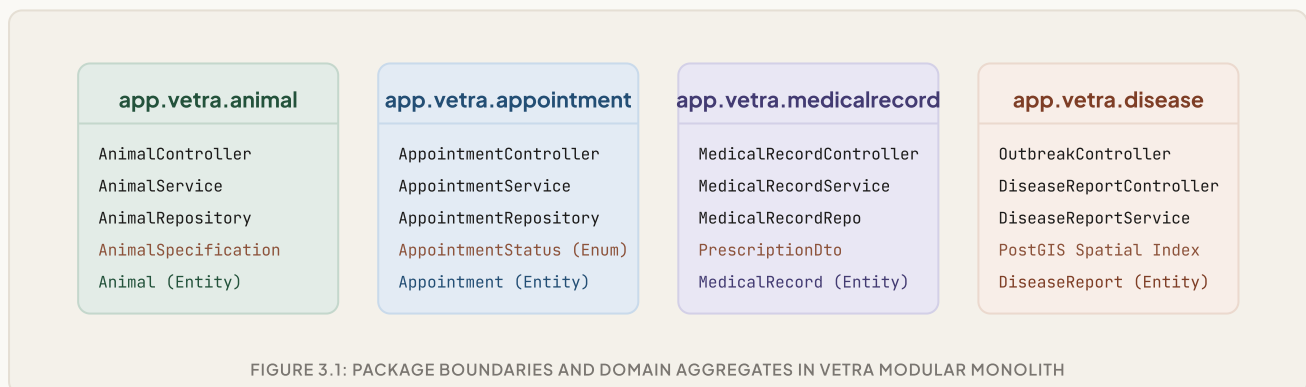
REST Controllers validate HTTP requests using JSR-380 annotations (`@Valid`) and map domain outputs to standardized DTO envelopes.

2. Application Services

Encapsulate business use cases, transactional boundaries (`@Transactional`), and cross-module business rule orchestrations.

3. Domain & Repositories

JPA Entities, PostGIS geometries, and dynamic Specification builders isolated behind repository interfaces.



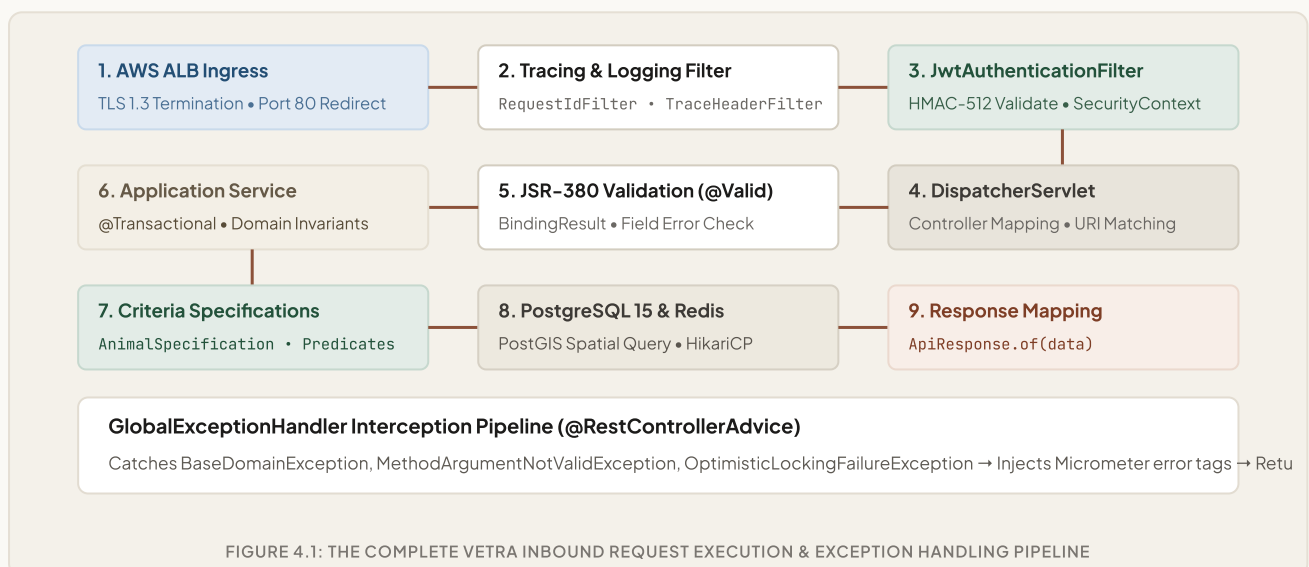
Shared Infrastructure Package (`app.vetra.infrastructure`)

Cross-cutting technical concerns are isolated inside `app.vetra.infrastructure`: security (JWT Filter & BCrypt), exception (`GlobalExceptionHandler`), cache (Redis configurations), and metrics (Prometheus & Micrometer Observation).

04 — REQUEST PIPELINE

End-to-End Request Execution Lifecycle

The precise 9-stage journey of an HTTP request from AWS ALB ingress to database persistence and JSON serialization.



Distributed Trace Correlation

TraceHeaderFilter extracts or generates W3C traceparent headers (traceId, spanId) and binds them into Logback's Mapped Diagnostic Context (MDC), ensuring 100% trace correlation across every log statement.

Transactional Isolation Level

Application services run with Spring's default ISOLATION_READ_COMMITTED and PROPAGATION_REQUIRED, guaranteeing that concurrent requests observe committed database states only.

05 — SECURITY INTERNALS

Authentication & Security Filter Chain

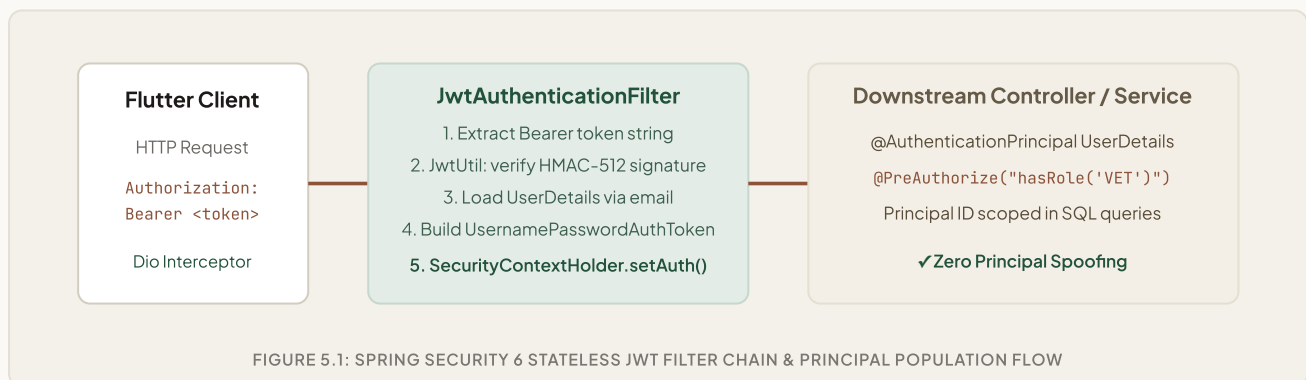
Stateless cryptographic JWT authentication, BCrypt password hashing, and Spring Security 6 FilterChain architecture.

SecurityFilterChain Configuration

- **CSRF**: Explicitly disabled (`AbstractHttpConfigurer::disable`) as the REST API is purely stateless.
- **Session Creation**: `SessionCreationPolicy.STATELESS` (no `HttpSession`).
- **Password Encoder**: `BCryptPasswordEncoder` (strength 12).
- **Exception Entry Point**: `CustomAuthenticationEntryPoint` formatting 401 Unauthorized errors as JSON envelopes.

Access Token (JWT) Anatomy

- **Algorithm**: HMAC-SHA512 via JJWT 0.12.6.
- **Subject (sub)**: User email address.
- **Custom Claims**: `userId` (UUID), `role` (`ROLE_FARMER` | `ROLE_VETERINARIAN`).
- **Expiration**: Configured in `JwtProperties` (default: 86,400s / 24 hours).



Public Endpoint Whitelist

Configured in `SecurityConfig.PUBLIC_ENDPOINTS`: `/api/v1/auth/farmer/register`, `/api/v1/auth/farmer/login`, `/api/v1/auth/vet/register`, `/api/v1/auth/vet/login`, `/api/v1/auth/refresh`, `/api/v1/auth/logout`, `/actuator/health/**`, and `/v3/api-docs/**`. All other paths require authenticated bearer tokens.

06 — TOKEN GOVERNANCE

Refresh Token Rotation & Revocation Engine

Single-use cryptographically random refresh token rotation and database-backed session termination.

Refresh Token Schema (refresh_tokens)

- **Token Value:** Cryptographically secure 256-bit random UUID string.
- **State Flags:** revoked (BOOLEAN), expiry_date (TIMESTAMP).
- **Entity Link:** Belongs to User (user_id foreign key).
- **Rotation Invariant:** When consumed, the old token is marked revoked and a fresh token is issued.

Revocation & Logout Invariant

Calling POST /api/v1/auth/logout sets revoked = TRUE on the active refresh token. Subsequent attempts to rotate with that token fail immediately with HTTP 401 Unauthorized, terminating compromised sessions.

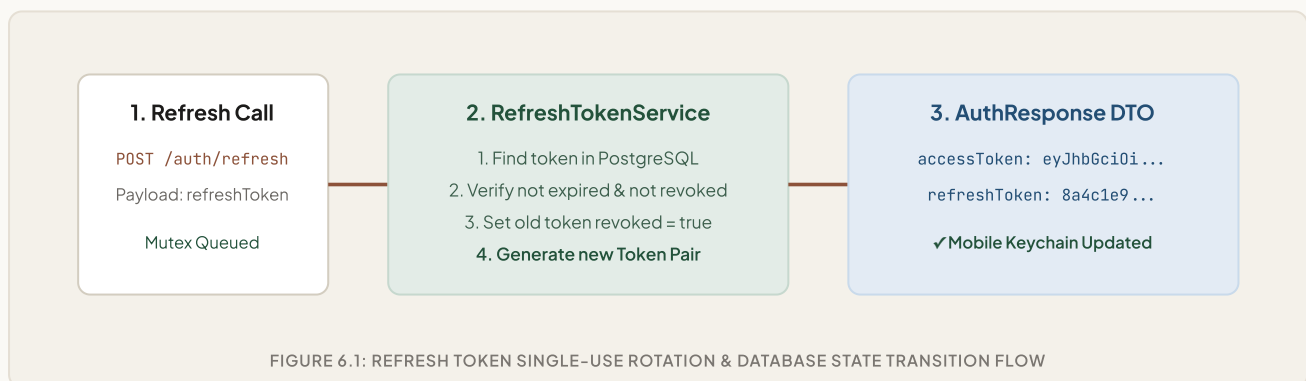


FIGURE 6.1: REFRESH TOKEN SINGLE-USE ROTATION & DATABASE STATE TRANSITION FLOW

Mobile Token Desynchronization Prevention

In the Flutter client, AuthInterceptor wraps token refresh calls behind an asynchronous Completer mutex queue. When multiple concurrent requests encounter 401 Unauthorized, only the first triggers POST /api/v1/auth/refresh; all other queued requests await the single rotation result and replay transparently.

07 — AUTHORIZATION

Multi-Tenant Isolation & Role-Based Access

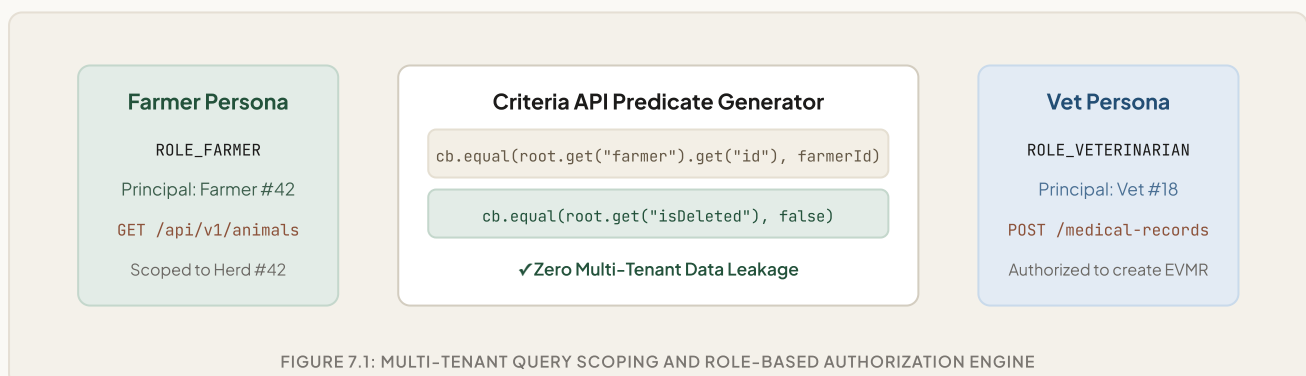
Principal-based SQL predicate scoping and Spring Security declarative method authorization.

Role-Based Access Control (RBAC)

- **ROLE_FARMER**: Register animals, book clinical appointments, view own herd passports and medical history.
- **ROLE_VETERINARIAN**: Accept appointments, record immutable clinical EVMR entries, issue prescriptions, and submit disease reports.
- **Enforcement**: `@EnableMethodSecurity(prePostEnabled = true)` with `@PreAuthorize` guards on controllers.

Tenant Data Isolation Principle

Data queries never rely on client-supplied farmer IDs in request bodies. The backend extracts the authenticated `userId` from `SecurityContextHolder` and injects it into JPA Criteria Predicates (`farmer.id = :farmerId`). Cross-tenant queries return empty result sets rather than unauthorized data.



Unauthorized Resource Access Handling

If an authenticated farmer attempts to access an animal or record belonging to another tenant (`animal.getFarmer().getId() != principal.getId()`), the service immediately throws `UnauthorizedResourceAccessException` which maps to HTTP 403 Forbidden.

08 — API CONTRACTS

REST API: Authentication & User Profiles

Endpoint specifications, JSON payloads, and validation constraints for user onboarding and credential exchange.

METHOD	ENDPOINT	REQUEST / VALIDATION INVARIANTS	RESPONSE PAYLOAD
POST	/api/v1/auth/farmer/register	FarmerRegisterRequest: email, password (min 8 chars), fullName, farmName, location.	AuthResponse (tokens + profile)
POST	/api/v1/auth/vet/register	VetRegisterRequest: email, password, fullName, licenseNumber, clinicAddress, specialization.	AuthResponse (tokens + profile)
POST	/api/v1/auth/farmer/login	LoginRequest: email, password (BCrypt verification).	AuthResponse (tokens + profile)
POST	/api/v1/auth/vet/login	LoginRequest: email, password (BCrypt verification).	AuthResponse (tokens + profile)
POST	/api/v1/auth/refresh	RefreshTokenRequest: valid, non-revoked refresh token string.	AuthResponse (new token pair)
POST	/api/v1/auth/logout	RefreshTokenRequest: revokes refresh token in database.	ApiResponse<Void> (200 OK)

Representative Request: Farmer Login

```
POST /api/v1/auth/farmer/login Content-Type:
application/json { "email":
"farmer.ramesh@vetra.app", "password":
"SecurePassword123!" }
```

Representative Response: AuthResponse

```
HTTP/1.1 200 OK Content-Type: application/json {
  "success": true, "statusCode": 200, "message":
  "Authentication successful", "data": { "accessToken":
  "eyJhbGciOiJIUzUxMiIsInR5cCI6IkpzZW50L3N1bWU6PSJ3c9e6679-
  7425-40de-944b-e07cf1f90ae7", "tokenType": "Bearer",
  "expiresIn": 86400, "user": { "id": "c5946b52-df21-
  419b-a7eb-6c1eb2ea7231", "email":
  "farmer.ramesh@vetra.app", "role": "ROLE_FARMER",
  "fullName": "Ramesh Patil" } } }
```

09 — API CONTRACTS

REST API: Animal Passport & Dynamic Search

Endpoint contracts, unique QR identification, and multi-parameter dynamic Specification search.

METHOD	ENDPOINT	PURPOSE & PARAMETERS	AUTH / SCOPE
POST	/api/v1/animals	Register animal: tagNumber, species, gender, dob, breed, name.	ROLE_FARMER
GET	/api/v1/animals	List all animals owned by authenticated farmer.	ROLE_FARMER
GET	/api/v1/animals/{id}	Get animal details by UUID primary key.	AUTHENTICATED
GET	/api/v1/animals/tag/{tagNumber}	Lookup animal by unique ear tag identifier.	AUTHENTICATED
GET	/api/v1/animals/qr/{qrCodeId}	Digital Animal Passport lookup via scanned QR code.	AUTHENTICATED
GET	/api/v1/animals/search	Dynamic search: tagNumber, species, gender, name, breed.	ROLE_FARMER

Dynamic Search Request Parameters

```
GET /api/v1/animals/search?
species=CATTLE&gender=FEMALE&page=0&size=20
Authorization: Bearer eyJhbGciOiJIUzUxMiIs... #
Dynamic Criteria Engine executes: SELECT a FROM
Animal a WHERE a.farmer.id = :farmerId AND
a.isDeleted = false AND a.species = 'CATTLE' AND
a.gender = 'FEMALE' ORDER BY a.createdAt DESC
```

Animal DTO Response Structure

```
{ "success": true, "statusCode": 200, "data": { "id":
"e4a3b129-87c2-4821-b3fa-9f829a3e2101", "tagNumber":
"MH-18-CT-9042", "name": "Gauri", "species":
"CATTLE", "breed": "Gir", "gender": "FEMALE",
"dateOfBirth": "2022-04-15", "qrCodeId": "QR-MH18-
9042-VETRA", "healthStatus": "HEALTHY", "createdAt":
"2026-08-16T10:00:00Z" } }
```

10 — STATE MANAGEMENT

REST API: Appointments & State Machine

Clinical appointment booking lifecycle, status state machine, and optimistic locking concurrency control.

Appointment State Machine Invariants

- REQUESTED: Farmer creates visit request for an animal.
- CONFIRMED: Veterinarian accepts and schedules date/time.
- IN_PROGRESS: Clinical triage underway at farm/clinic.
- COMPLETED: Finalized upon creation of medical record.
- CANCELLED: Allowed only from REQUESTED or CONFIRMED.

Optimistic Locking Concurrency Control

Appointment entities include a `@Version Long version` column. If a farmer cancels while a veterinarian confirms simultaneously, Hibernate throws `OptimisticLockingFailureException`, caught by `GlobalExceptionHandler` as HTTP 409 Conflict.

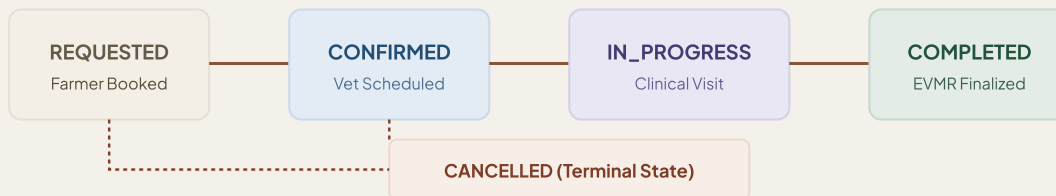


FIGURE 10.1: CLINICAL APPOINTMENT LIFECYCLE & STATE MACHINE TRANSITIONS

METHOD	ENDPOINT	PAYLOAD / STATE TRANSITION	AUTH / SCOPE
POST	/api/v1/appointments	Create appointment: animalId, vetId, scheduledTime, notes.	ROLE_FARMER
GET	/api/v1/appointments	List appointments (scoped to current farmer or veterinarian).	AUTHENTICATED
PATCH	/api/v1/appointments/{id}/status	Transition status: {"status": "CONFIRMED"}.	AUTHENTICATED

11 — CLINICAL DOCUMENTATION

REST API: Immutable Medical Records (EVMR)

Veterinarian clinical entry contracts, prescription lineage, and immutable electronic health history.

Clinical Record Invariants

1. **Veterinarian-Only Authoring:** Enforced via `@PreAuthorize("hasRole('VETERINARIAN')")`.
2. **Appointment Linkage:** Every record references a valid Appointment and transitions its state to COMPLETED.
3. **Immutable History:** Once created, clinical entries cannot be modified or deleted, preserving medical and legal auditability.

Prescription & Withdrawal Tracking

Prescriptions record drug names, dosages, durations, and mandatory **withdrawal periods** (days required before milk/meat can enter the human food chain), preventing Antimicrobial Resistance (AMR).

METHOD	ENDPOINT	REQUEST / PARAMETERS	AUTH / SCOPE
POST	/api/v1/medical-records	CreateMedicalRecordRequest: appointmentId, diagnosis, treatment, prescriptions.	ROLE_VETERINARIAN
GET	/api/v1/medical-records/{id}	Get medical record details by record UUID.	AUTHENTICATED
GET	/api/v1/medical-records/animal/{animalId}	Chronological medical history for digital animal passport.	AUTHENTICATED

Representative Request: Create Medical Record

```
POST /api/v1/medical-records Content-Type:
application/json { "appointmentId": "a1b2c3d4-...",
"diagnosis": "Acute Bovine Mastitis", "treatment":
"Intramammary antibiotic infusion", "prescriptions":
[ { "medicineName": "Amoxicillin Trihydrate",
"dosage": "15 mg/kg daily", "durationDays": 5,
"withdrawalDays": 7 } ] }
```

Immutable Record Response Envelope

```
HTTP/1.1 201 Created Content-Type: application/json {
"success": true, "statusCode": 201, "message":
"Medical record created successfully", "data": {
"id": "m9876543-21ef-...", "animalId":
"e4a3b129-...", "veterinarianName": "Dr. Sneha
Kulkarni", "diagnosis": "Acute Bovine Mastitis",
"treatment": "Intramammary infusion", "createdAt":
"2026-08-16T11:30:00Z" } }
```

12 — GEOSPATIAL INTELLIGENCE

REST API: Disease Surveillance & Spatial Radar

PostGIS geospatial point indexing, outbreak detection, and proximity queries (ST_DWithin).

Geospatial Data Modeling (PostGIS)

- **Coordinates:** Stored as GEOMETRY(Point, 4326) in disease_reports and outbreaks.
- **Spatial Indexes:** Accelerated via PostGIS GiST spatial indexes.
- **Proximity Calculation:** Uses spheroidal distance calculation (ST_DWithin(location, ST_MakePoint(lng, lat), radius)).

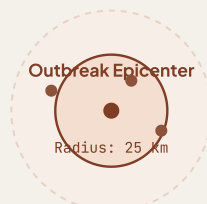
Real-Time Outbreak Clustering

When multiple disease reports of the same infectious pathogen occur within a defined radius and temporal window, the backend flags a spatial cluster and triggers outbreak containment telemetry.

METHOD	ENDPOINT	SPATIAL QUERY PARAMETERS	RESPONSE PAYLOAD
POST	/api/v1/disease-reports	Submit report: diseaseName, species, latitude, longitude, affectedCount.	DiseaseReportDto
GET	/api/v1/disease-reports	List reports with optional filters (species, status, date range).	Page<DiseaseReportDto>
GET	/api/v1/outbreaks/active	Spatial query: latitude, longitude, radiusKm (default: 25km).	List<OutbreakDto>
GET	/api/v1/outbreaks/spatial-clusters	Aggregated density clusters for map heatmap rendering.	List<ClusterDto>

PostGIS Spatial SQL

```
SELECT * FROM outbreaks o
WHERE ST_DWithin(
  o.center_location,
  ST_MakePoint(:lng,:lat),
  :radiusMeters)
```



Mobile Radar Output

- ✓ Foot & Mouth Disease Alert
- ✓ Distance: 12.4 km South-East
- ✓ Biosecurity Protocol Sent

FIGURE 12.1: POSTGIS SPATIAL OUTBREAK RADIUS QUERY AND MOBILE RADAR ALERT EXECUTION

13 — DYNAMIC QUERY ENGINE

Dynamic JPA Criteria API & Specifications

How `AnimalSpecification.java` solved PostgreSQL prepared statement parameter deduction failures.

The Legacy Static JPQL Failure

A static JPQL query (`:species IS NULL OR a.species = :species`) produced `PSQLException: could not determine data type of parameter $X` on PostgreSQL RDS when optional enum or UUID parameters were null.

The Dynamic Criteria Solution

Using `JpaSpecificationExecutor`, `AnimalSpecification` builds predicates dynamically at runtime. If a search filter is null or blank, no predicate is added, generating clean SQL with 0 untyped parameters.

Source Code: `AnimalSpecification.java` Implementation

```
public class AnimalSpecification { public static Specification<Animal> filterAnimals( UUID farmerId, String
tagNumber, Species species, AnimalGender gender, String name, String breed) { return (Root<Animal> root,
CriteriaQuery<?> query, CriteriaBuilder cb) → { List<Predicate> predicates = new ArrayList<>(); // Mandatory multi-
tenant isolation predicate if (farmerId ≠ null) { predicates.add(cb.equal(root.get("farmer").get("id"), farmerId));
} predicates.add(cb.equal(root.get("isDeleted"), false)); // Dynamic optional predicates – appended ONLY when non-
null if (tagNumber ≠ null && !tagNumber.trim().isEmpty()) { predicates.add(cb.like(cb.lower(root.get("tagNumber")),
"%" + tagNumber.trim().toLowerCase() + "%")); } if (species ≠ null) { predicates.add(cb.equal(root.get("species"),
species)); } if (gender ≠ null) { predicates.add(cb.equal(root.get("gender"), gender)); } if (name ≠ null &&
!name.trim().isEmpty()) { predicates.add(cb.like(cb.lower(root.get("name")), "%" + name.trim().toLowerCase() + "%"));
} return cb.and(predicates.toArray(new Predicate[0])); }; }
```

Compile-Time Type Safety

Criteria API provides compile-time checking of entity attributes, eliminating runtime SQL syntax errors and enabling seamless schema refactoring across domain entities.

Live AWS RDS Staging Verification

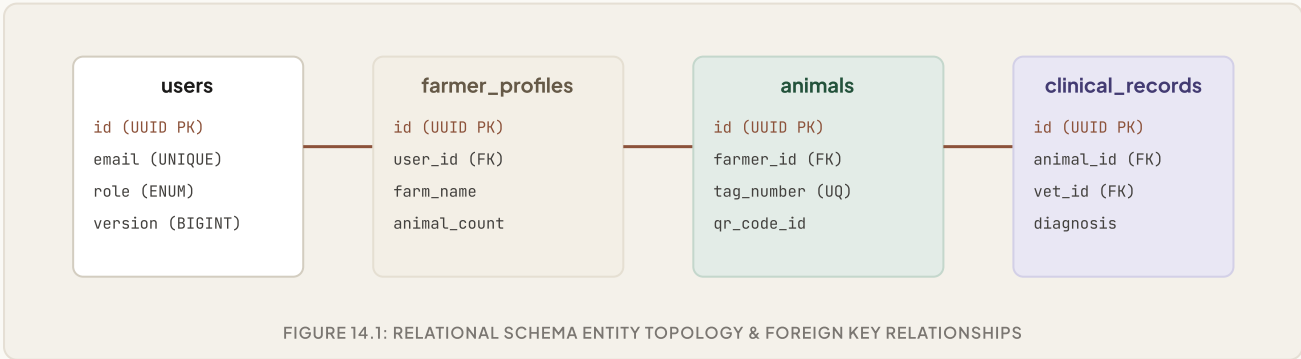
Verified live on Amazon RDS PostgreSQL 15.13 across all 6 search permutations: Tag-only, Tag+Species, Tag+Gender, Species+Gender, All filters, and No filters. 100% tests returned 200 OK.

14 — PERSISTENCE LAYER

Relational Schema & Flyway Migrations

PostgreSQL 15 relational entities, UUID v4 primary keys, and 14 versioned Flyway database migrations.

MIGRATION	TABLES & EXTENSIONS CREATED	KEY CONSTRAINTS, INDEXES & PURPOSE
V1__extensions	uuid-oss, postgis	Enables database-level UUID v4 generation and spatial geometry types.
V2__schema	users, farmer_profiles, vet_profiles	User authentication credentials, BCrypt hashes, user roles, profile linkage.
V3__animals	animals	Livestock identities, unique tag_number, species, breed, QR identifiers.
V4__appointments	appointments	Clinical visit booking, status enums, scheduled timestamps, optimistic lock version.
V5__records	medical_records, prescriptions	Immutable clinical diagnoses, treatments, drug dosages, withdrawal tracking.
V8__ai_scans	ai_scans	Diagnostic image scan records with veterinarian_verified invariant.
V11__surveillance	disease_reports, outbreaks	Geospatial coordinates (Point, 4326) with PostGIS GiST spatial indexes.
V14__refresh_tok	refresh_tokens	Cryptographic 256-bit refresh token persistence with revocation audit flags.



15 — PERFORMANCE

Distributed Caching & Invalidation (Redis 7)

Multi-region TTL caching, serialization envelopes, and proactive invalidation via Spring Cache.

ANIMAL_PASSPORT (30m)

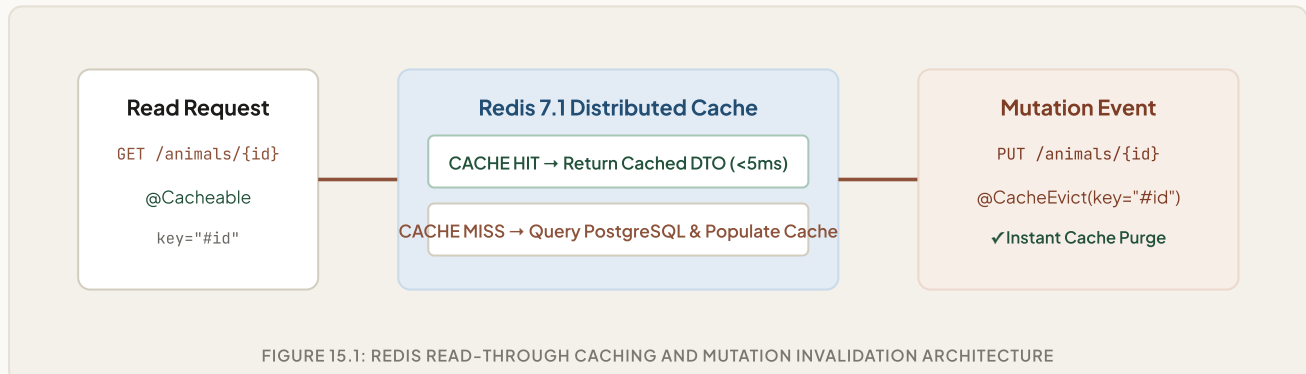
Caches full animal profile and vaccination history. Evicted on animal update or new medical record creation.

VET_PROFILE (1h)

Caches verified veterinary credentials, clinic address, and active availability slots for fast booking lookups.

DASHBOARD_STATS (15m)

Caches aggregated herd counts, upcoming visits, and active local alerts, reducing heavy database aggregation queries.

**Redis Serialization & TLS Security**

Configured via `RedisConfig.java`: Redis keys are serialized with `StringRedisSerializer`, and values are serialized with `GenericJackson2JsonRedisSerializer` with ISO-8601 timestamp support. In production, Redis operates with mandatory in-transit TLS encryption.

16 — ERROR ARCHITECTURE

Exception Handling & Standardized Envelopes

Typed domain exception taxonomy, RFC 7807 problem details alignment, and Micrometer span tagging.

Standardized Response Envelope (`ApiResponse<T>`)

Every HTTP response conforms to a deterministic schema:

- `success` (`BOOLEAN`): Indicates execution status.
- `statusCode` (`INT`): Canonical HTTP status code.
- `message` (`STRING`): Human-readable status or error summary.
- `data` (`T`): Strongly-typed payload (null on errors).
- `errors` (`List<FieldError>`): Validation error map.
- `timestamp` (`Instant`): ISO-8601 server timestamp.

Domain Exception Hierarchy

- `ResourceNotFoundException` → HTTP 404 Not Found.
- `ConflictException` → HTTP 409 Conflict.
- `UnauthorizedResourceAccessException` → HTTP 403 Forbidden.
- `BusinessRuleException` → HTTP 422 Unprocessable Entity.
- `MethodArgumentNotValidException` → HTTP 400 Bad Request.

Standard RFC 7807 Validation Error Envelope

```
HTTP/1.1 400 Bad Request Content-Type: application/json { "success": false, "statusCode": 400, "message": "Request validation failed", "data": null, "errors": [ { "field": "tagNumber", "message": "Ear tag number cannot be blank" }, { "field": "species", "message": "Species must be one of: CATTLE, BUFFALO, SHEEP, GOAT" } ], "timestamp": "2026-08-16T12:00:00Z" }
```

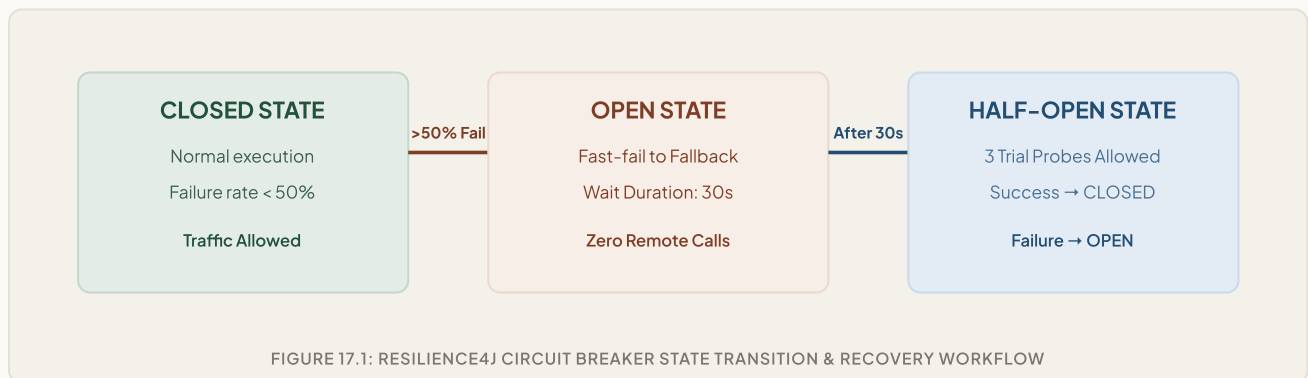
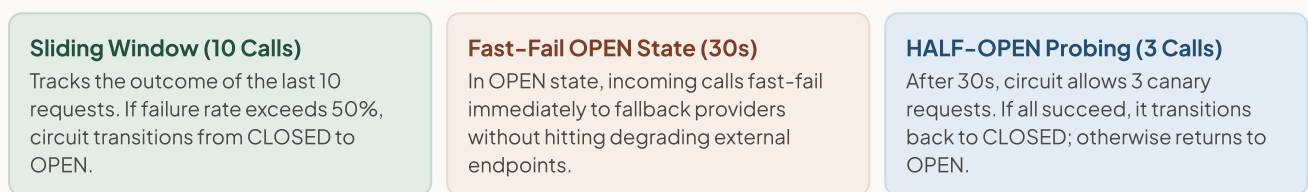
Micrometer Span Error Tagging

`GlobalExceptionHandler` automatically attaches error events and tags to the active Micrometer span when an exception occurs, ensuring that failed requests surface instantly in Grafana Tempo tracing dashboards with complete stack trace context.

17 — FAULT TOLERANCE

Reliability Engineering & Circuit Breakers

Resilience4j 2.4.0 circuit breaker state transitions, timeout thresholds, and graceful fallback execution.



Fallback Provider Implementation

When external AI diagnosis or image upload services fail, `FaiLoverManager` routes requests to a local rule-based fallback provider, ensuring the client receives a structured response rather than an unhandled 500 error.

18 — TELEMETRY

Observability, Metrics & Distributed Tracing

Spring Boot Actuator probes, Prometheus metrics, and OpenTelemetry distributed tracing.

Liveness & Readiness

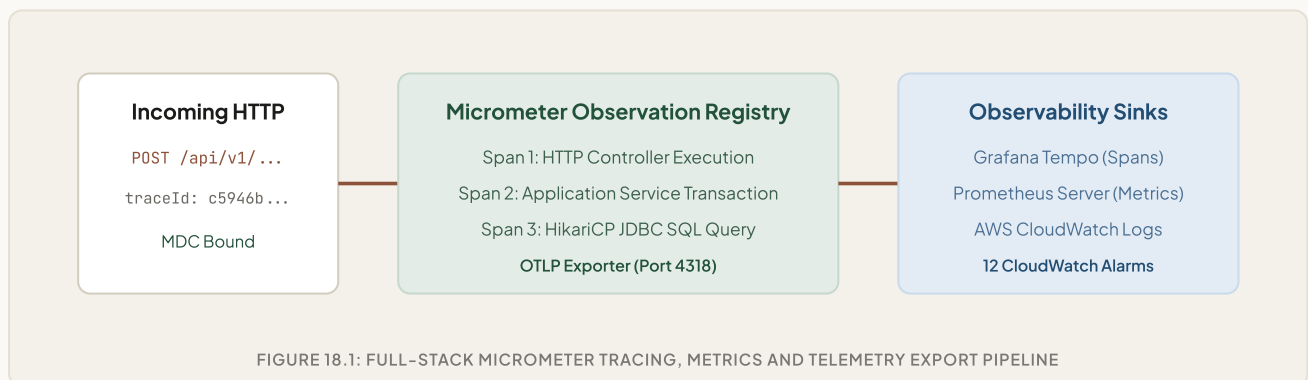
/actuator/health/liveness confirms internal JVM health.
/actuator/health/readiness probes PostgreSQL & Redis connectivity for ALB traffic draining.

Prometheus Telemetry

/actuator/prometheus exports HTTP request rates, JVM memory pool allocations, garbage collection pauses, and HikariCP connection pool metrics.

OpenTelemetry Tracing

micrometer-tracing-bridge-otel automatically creates spans for HTTP requests, Spring Security filters, and JDBC database queries.



HikariCP JDBC Connection Tracing

Configured via datasource-micrometer-spring-boot:1.0.6: every SQL query executed by Hibernate is automatically instrumented as a child span within the active HTTP trace, showing exact SQL query strings and execution latencies in Grafana Tempo.



VETRA

PUBLICATION SERIES • DOCUMENT 02

SECTION 19 • CLOSING COLOPHON

ENGINEERING PUBLICATION SERIES • DOCUMENT 02

VETRA: API & Backend Architecture

Engineering the Core Veterinary Operations Platform

Built & Engineered by Om Rajput

B.Tech in Computer Science & Engineering (Data Science)

SVKM's Institute of Technology, Dhule

"Clean Architecture, Relational Integrity & Resilient Cloud Systems."

Technical Verification Summary

253/253 Backend Tests Passed • 0 Checkstyle Violations • PostgreSQL 15 & Redis 7 Active on Staging.

BUILD HASH: VETRA-BACKEND-DOC02-

2026

PAGE 20 / 20